CASE STUDY: SaigonBite – Cloud-Kitchen Food Delivery Hub

Course: Software Engineering | Semester: Sem01_2627 (Semester 1, 2026-2027)

1. Context & Background

Ho Chi Minh City is one of the busiest food-delivery markets in Southeast Asia. **SaigonBite** is a planned network of **cloud kitchens** ("hubs") in districts such as District 1, Binh Thanh and Thu Duc. A hub is a delivery-only kitchen that cooks for several **virtual brands** (for example "Pho Express", "Banh Mi Corner" and "Green Bowl"). A customer picks a brand in one app, the hub cooks the order, and a shipper delivers it to a home, office or hotel.

Unlike a simple shop, an order passes through **three roles** (customer, kitchen staff, shipper) and **several states** before it is complete. The city and its hub operators also face two practical problems:

- **Two customer groups, one platform.** Local Customers pay with local e-wallets (MoMo, VNPay) or cash on delivery and register with a Vietnamese phone number. Foreign Tourists need international options (Apple Pay, PayPal) or cash, and register with a passport number and an email address because they often have no local phone.
- **Uneven kitchen load.** Hubs are overloaded at lunch and dinner while others sit idle. The operator wants pricing that nudges demand toward quiet kitchens and protects busy ones.

2. Development Challenges

As second-year software engineering students, tackling this system presents several core technical and design challenges:

- **Order Lifecycle State Management:** An order moves through Placed, Confirmed, Preparing, Ready, Out for Delivery, Delivered, and the terminal states Cancelled and Failed. Only valid transitions by the correct role may be allowed.
- **Layered Pricing Logic:** The total depends on the item subtotal, a hub-load discount, a voucher with a cap, a distance-based delivery fee, a rush-hour surcharge, a free-delivery threshold and rounding. The **order of calculation** matters and must be specified and tested.
- **Architectural Separation of Concerns:** Designing a multi-gateway payment system without "spaghetti code". We must use the **Strategy Design Pattern** so that local and international payments are handled cleanly, and pricing rules must not leak into the UI or controllers.
- **Actor Generalization:** Different registration and verification flows for Local Customers and Foreign Tourists inside the same database schema.
- **Coping with Change:** Real clients change their minds. A **change request** will be issued in Week 5, and a good design will absorb it with small, local changes.

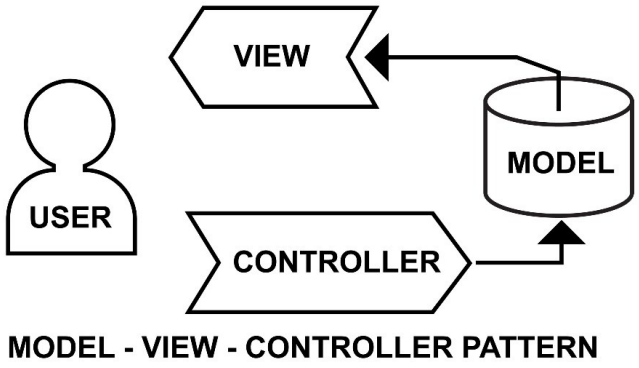
3. Project Vision & Objectives

Vision: To engineer a unified, scalable and user-friendly food-delivery platform that integrates hub and menu management, load-aware dynamic pricing, and localized and international payment processing.

Key Objectives:

1. **Full CRUD Implementation:** Complete Create, Read, Update and Delete functionality for core entities (Users, Hubs, Menu Items, Vouchers, Orders). Where business history must be preserved, "Delete" may be a justified soft-delete.
2. **Load-Aware Incentive:** Implement the "Quiet Kitchen" discount (10% off the food subtotal when the chosen hub is under 30% load) and the hub protection rule (no new orders at 90% load or above).
3. **Robust Quality Assurance:** Apply Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) to **every numeric threshold** in the pricing algorithm.
4. **Actionable Reporting:** Provide System Admins with at least two distinct operational reports (Revenue and Hub Utilization).
5. **End-to-End Traceability:** Every requirement carries an ID that is cited in the design, the code and the tests, so that the Requirements Traceability Matrix (RTM) grows throughout the project instead of being written at the end.

4. Overall Project Requirements



Model-View-Controller pattern (AI generated)

A. Functional Requirements (headline level)

The headline requirements below are expanded into numbered **Business Rules (BR1 to BR18)** with exact values in the separate "Project Requirements" document. Your own FR list in Part 1 must refine them and use IDs.

ID	Requirement
FR1	The system shall let a customer build an order from the menus of one hub and shall calculate the total price (items, discounts, delivery fee, surcharges) according to the business rules.
FR2	The system shall apply a 10% "Quiet Kitchen" discount to the item subtotal when the selected hub is currently under 30% load.
FR3	The system shall refuse new orders for a hub that is at or above 90% load and shall ask the customer to choose another hub.

ID	Requirement
FR4	The system shall allow Local Customers to pay with MoMo, VNPay or Cash on Delivery.
FR5	The system shall allow Foreign Tourists to pay with Apple Pay, PayPal or Cash on Delivery.
FR6	The system shall enforce the order lifecycle: kitchen staff and shippers update the status, only valid transitions are accepted, and a customer can track and (when allowed) cancel an order.
FR7	The system shall let Admins manage vouchers and shall accept at most one valid voucher per order.
FR8	The system shall register Local Customers by Vietnamese phone number and Foreign Tourists by passport number and email.
FR9	The system shall generate at least two administrative reports (Revenue and Hub Utilization).

B. Non-Functional Requirements (Quality Attributes)

- **Usability:** The UI must be responsive (Bootstrap or an equivalent front-end framework) and support bilingual toggling (English/Vietnamese).
- **Performance:** Price calculation, including discounts and surcharges, must complete in **under 1.5 seconds** for an order of up to 30 line items.
- **Maintainability:** The codebase must strictly follow a 3-Tier or MVC architecture. Adding a new payment method must require **adding one new class** and registering it, without editing the existing payment classes.
- **Security:** Passwords are stored hashed, and each role (Customer, Kitchen Staff, Shipper, Admin) can only reach its own functions.
- **Team-defined:** Add at least two further measurable quality attributes of your own in Part 1 (for example reliability or auditability).

5. Scoring Criteria (The "Tiered" Target)

Our team must self-select a technological tier, which caps our maximum possible grade based on technical complexity. We are evaluated on a 100% scale across 4 phases:

- **Tier 1 (Max 7.0):** C# WinForms (3-Tier)
- **Tier 1B (Max 7.5):** Core PHP / MySQL (MVC)
- **Tier 2 (Max 8.5):** ASP.NET Web Forms/MVC Database-First
- **Tier 3 (Max 9.5):** ASP.NET MVC Code-First + Entity Framework
- **Tier 4 (Max 10.0):** Tier 3 + **all three** of: live sandbox API integration (VNPay or PayPal sandbox), an interactive Admin dashboard, and automated tests. Completing only some of the three stays within the Tier 3 cap.

Weight Distribution:

- Part 1 (20%): Requirements Doc (15%) + Solution Pitch (5%)
- Part 2 (25%): Design Doc & Figma (20%) + Design Review (5%)

- Part 3 (35%): Code, GitHub, QA Testing (30%) + Live Demo (5%)
- Final Submission (20%): Consolidated Report with Traceability/Lessons Learnt (10%) + Video Demo (10%)

Marks are awarded to the team and then adjusted for each member by a **Contribution Factor** based on commit history, the peer review and individual Q&A (details in the Project Requirements document).

6. Necessary Actions & SDLC Roadmap



Software Development Life Cycle phases (AI generated)

To successfully deliver this software, our team will execute the following actions across a 7-week Agile timeline:

Weeks 1-2 (Analysis)

- Assign one main use case (with full CRUD) to each team member. One of them must be "Place and Manage Orders".
- Draft the Proposal, FRs (with IDs), NFRs and an **Ambiguity Log**: the specification deliberately leaves some details open, and the team must record its assumptions.

Weeks 3-4 (Design)

- Create CRC Cards for object planning.
- Draw the UML Use Case, Sequence (or Activity), Class diagrams, and a **State diagram for the Order lifecycle**.
- Show the Strategy pattern for payments in the class design.
- Design a 4-stage interactive Figma prototype (Sketch -> Prototype) using Bootstrap grid layouts.

Weeks 5-6 (Implementation & QA)

- **Week 5 checkpoint:** show a running skeleton (database schema plus one use case working end to end). The instructor issues the **Change Request** this week.
- Code the application with strict MVC/3-Tier separation and commit regularly to GitHub (more than 10 meaningful commits per student).

- Write a Test Plan and execute unit tests on the pricing logic using EP and BVA; test invalid order-state transitions.
- Maintain the Requirements Traceability Matrix (RTM), write a one-page Change Request Impact Note, compile the final PDF report, record the video demo, and present the live working product.

(Please refer to the separate "Project Requirements" document for detailed grading rubrics, submission guidelines, business rules and specific deliverables for each tier.)